

Design Justification Report

INT8 Autoencoder Anomaly-Detection Co-Processor (sky130A)
ECE 410/510 HW4AI · Milestone 4 · Final Submission

Abstract. This report documents a custom INT8 co-processor that accelerates the forward pass of a four-layer autoencoder ($32 \rightarrow 16 \rightarrow 8 \rightarrow 16 \rightarrow 32$) plus a mean-squared-error (MSE) reconstruction-error threshold, for real-time anomaly detection on streaming sensor data. The design uses a time-multiplexed eight-MAC compute core, an AXI4-Stream data path with a configuration port for weights and threshold, and targets the SkyWater sky130A open PDK. It was implemented through the complete OpenLane 2 RTL-to-GDS flow: the final layout is DRC- and LVS-clean, closes timing at 40 MHz at the typical and fast corners, dissipates 0.269 W, and performs one inference in 5.43 μ s.

1. Problem and Motivation

The target application is real-time anomaly detection at the network edge, where a stream of sensor readings must be screened one sample at a time and a decision returned with low latency and within a tight power budget. The detection model is an undercomplete autoencoder: a 32-dimensional sensor vector is compressed through hidden layers of 16 and 8 units and reconstructed back to 32 dimensions, after which the mean-squared error between the input and its reconstruction is compared against a threshold. A large reconstruction error indicates an input the network was not trained to represent — i.e. an anomaly.

Profiling the reference model shows that the computation is overwhelmingly dominated by the linear layers' matrix-vector products: of the 2,657 floating-point-equivalent operations per inference, 2,560 (96%) come from the 1,280 multiply-accumulate (MAC) operations across the four layers, with the remaining 97 attributable to the MSE reduction. This is the kernel worth accelerating in hardware.

A custom accelerator is justified by the *shape* of the workload rather than raw throughput. The edge use case processes a single sample at a time as it arrives, so the batched throughput at which a CPU or GPU excels is irrelevant; what matters is per-sample latency, memory footprint, and energy per inference. A small fixed-function INT8 datapath can hold the entire model on-chip, respond to each sample in microseconds, and operate at a fraction of a watt — properties a general-purpose processor cannot match for this single-sample, always-on scenario.

2. Roofline Analysis

The arithmetic intensity of the kernel determines whether the design is limited by compute or by data movement. Because the model is small enough to keep all 1,352 INT8 weights resident on-chip, they are loaded once and reused across every inference; the only off-chip traffic in steady state is the 32-byte input sample and the 32-byte reconstruction (or, in practice, the single result word). Under this weights-stationary model the operational intensity is 2,657 FLOPs / 64 bytes $\approx$ **42 FLOP/byte**, which places the kernel far to the right of the roofline ridge point — the design is firmly compute-bound, not memory-bound.

The hardware compute roof is set by the eight parallel MACs: $8 \text{ MACs} \times 2 \text{ FLOP} \times 40 \text{ MHz} = 0.64 \text{ GFLOPS}$. The measured operating point is 0.49 GFLOPS (Section 8), or about 77% of that roof, shown in Figure 3. The ~23% gap is not data starvation — the 256-bit streaming interface can deliver a full sample in a single cycle — but idle MAC cycles spent in the non-MAC FSM states (weight/address setup, bias, activation copy, MSE, and done). Of the 217 cycles per inference, only 160 are MAC cycles ($1,280 \text{ MACs} / 8$), giving a MAC utilization of

74%, consistent with the measured fraction of roof. This analysis directly shaped the architecture: an eight-wide MAC array balances area against the compute the kernel needs, and keeping weights on-chip is what pushes the design into the compute-bound regime in the first place.

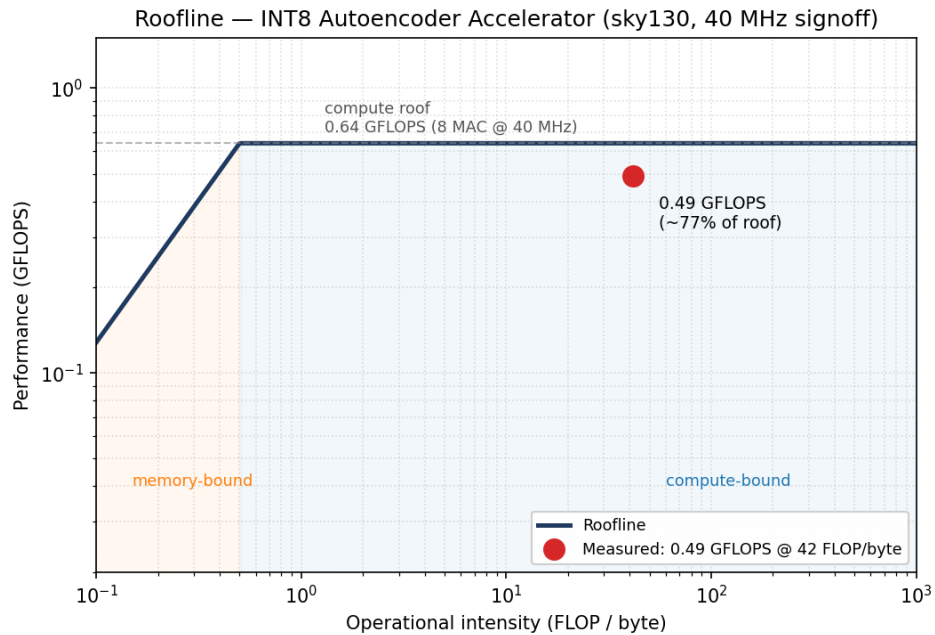


Figure 3 — Roofline. Measured operating point (0.49 GFLOPS at 42 FLOP/byte) against the 0.64 GFLOPS eight-MAC compute roof; the design is compute-bound at ~77% of roof.

3. Precision and Data Format

All weights and activations are 8-bit signed integers (INT8); MAC products accumulate into a 32-bit signed accumulator to avoid overflow across the up-to-32-term dot products. Each layer carries a per-tensor scale factor, computed at export as $\text{scale} = \max(|w|) / 127$, so that the INT8 range is used efficiently regardless of a layer's dynamic range. After accumulation the hardware requantizes the result with a fixed arithmetic right shift, applies ReLU, and clamps back to the INT8 range for the next layer. The MSE and threshold comparison are carried in a Q8.8 fixed-point format.

Naive post-training quantization of an FP32 model to INT8 loses accuracy. To avoid this the model was trained with quantization-aware training (QAT): in every forward pass during training the weights are fake-quantized to INT8 and back, so the network learns to compensate for rounding and the exported INT8 weights carry no post-training surprise. The effect is measurable on the detection task — the F1 score is 0.661 for the FP32 baseline and 0.672 for the INT8 + QAT model, i.e. quantization is not merely tolerated but accuracy is fully preserved. A subtle but important hardware detail in this section is signedness: the accumulator-to-activation requantization must use an arithmetic (sign-extending) shift and signed addition; an early bug in which an unsigned operand silently made the shift logical is documented in Section 9.

4. Dataflow and Architecture

The accelerator (Figure 1) consists of an interface module (interface.v) and a compute core (compute_core.v). The interface terminates the host-side AXI4-Stream and configuration ports and holds the result register; the compute core contains the eight-MAC engine, the 1,352-entry INT8 weight memory, the activation registers, and the control FSM with the MSE/threshold logic.

Figure 1 — System block diagram: host, AXI interface, and the 8-MAC compute core

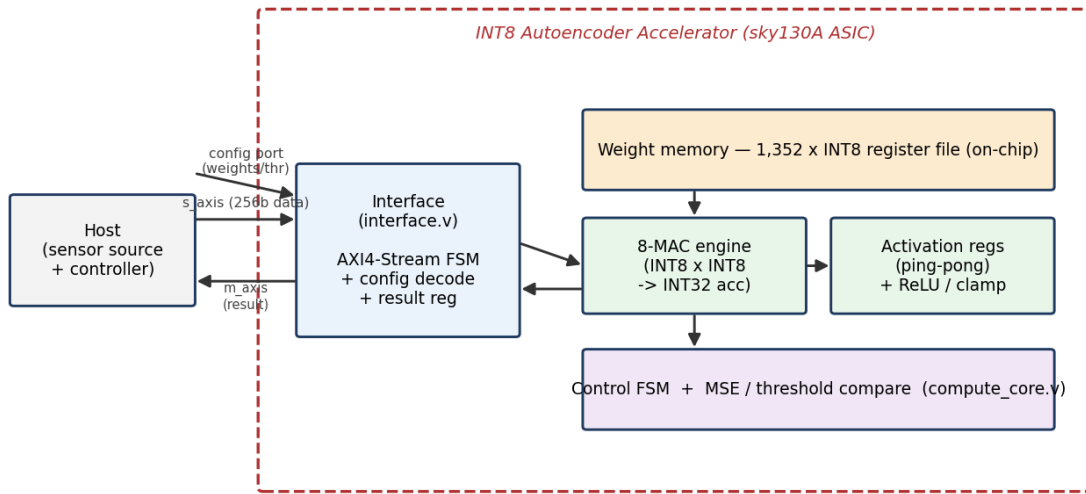


Figure 1 — System block diagram: host, AXI interface, and the on-chip eight-MAC compute core.

The dataflow actually implemented is a time-multiplexed, weight-streaming matrix-vector product. For each output neuron the eight MACs accumulate over the input vector eight elements per cycle, reading the corresponding weights from the on-chip register-file memory each cycle; activations are held in ping-pong registers so that one layer's outputs become the next layer's inputs, with ReLU and clamping applied in between. The control FSM (Figure 2) sequences the states `S_IDLE`, `S_LOAD`, `S_MAC`, `S_BIAS`, `S_COPY`, `S_MSE`, and `S_DONE`, looping the MAC/bias/copy group four times for the four layers. The `S_COPY` state was added to resolve a layer-transition race (Section 9) at a cost of four cycles, bringing the total to a fixed 217 cycles per inference, of which 160 are productive MAC cycles.

Figure 2 — Compute FSM and dataflow (time-multiplexed 8-MAC, weight-streaming MVM)

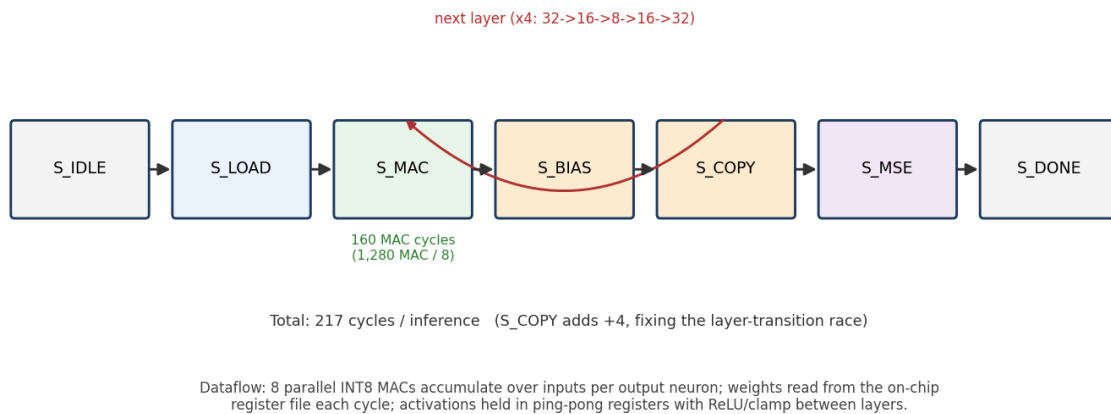


Figure 2 — Compute FSM and dataflow. The MAC/bias/copy group repeats per layer; total latency is a fixed 217 cycles per inference.

The single most consequential architectural decision is that the weight memory is realized as a flat register file rather than a compiled SRAM macro. This keeps the RTL simple and fully portable for simulation and verification, but it has large physical consequences — reading one of 1,352 entries per cycle synthesizes into a deep multiplexer tree, which dominates area and timing as discussed in Sections 7 and 9.

5. Hardware Interface

The host communicates through two channels. Sample data arrives over an AXI4-Stream slave port whose 256-bit data bus carries one complete 32-element INT8 sensor sample per beat, and results are returned over an AXI4-Stream master port. Weights and the detection threshold are written once through a lightweight AXI4-Lite-style configuration port with an address/data/enable handshake. This split matches the traffic pattern: the per-sample data stream is naturally a stream, while the one-time model configuration is naturally register-mapped.

The interface is not a bottleneck. At 40 MHz a 256-bit beat moves 32 bytes per cycle, a peak of 1.28 GB/s, whereas the design consumes only one 32-byte sample every 217 cycles — an average input demand of roughly 5.9 MB/s, more than two orders of magnitude below the interface's capacity. The accelerator therefore spends almost all of its time computing rather than waiting on I/O, which is precisely why the roofline analysis in Section 2 places it in the compute-bound region.

6. Verification

Verification proceeds at two levels. Unit testbenches exercise the compute core and the interface in isolation, and an end-to-end co-simulation (tb_top.v) drives the integrated design exclusively through its host-side AXI ports — loading the 1,352 trained weights and the threshold, streaming sensor samples, and reading back the anomaly flag — with no privileged access to internal signals. Correctness is judged against an *independent* Python INT8 reference model that reproduces the exact hardware arithmetic (per-layer scaling, fixed-shift requantization, INT8 clamp, ReLU), not against a prior run of the device itself.

On the trained-weight test the hardware classifies all eight labelled samples correctly: the four normal samples produce MSE values of 517–812, well below the 4,836 (Q8.8) threshold, and the four anomalies produce 8,860–9,770, well above it — 8/8 PASS, matching the reference. Figure 4 shows the end-to-end waveform: reset, the weight-load phase over the configuration port, then the eight samples streaming through the AXI interface with the sample counter advancing and the anomaly flag asserting.

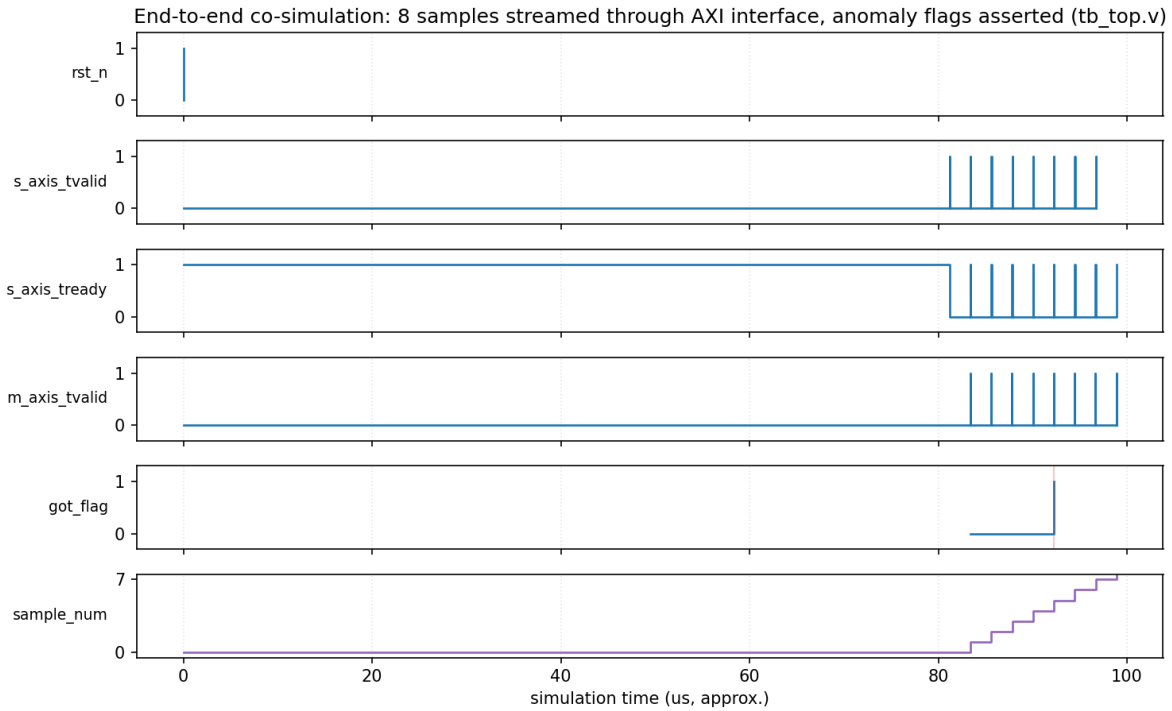


Figure 4 — End-to-end co-simulation waveform: reset, weight load, then eight samples streamed through the AXI interface with anomaly flags asserted (tb_top.v).

7. Synthesis Results

The design was carried through the complete OpenLane 2 / OpenROAD RTL-to-GDS flow on the sky130A high-density library, from synthesis to a signed-off layout. The 83 MHz target from pre-synthesis estimates proved infeasible in place-and-route (Section 9), so the implementation uses a 25 ns (40 MHz) clock on an $1800 \times 1800 \mu\text{m}$ die at 40% placement density. Synthesis maps the design to 48,945 cells / $682,088 \mu\text{m}^2$; the final placed layout contains 116,629 standard-cell instances and no macros.

Metric	Value
Die area	3.24 mm ² ($1800 \times 1800 \mu\text{m}$)
Placed standard-cell area	924,054 μm^2
Core utilization	29.1%
Total instances	116,629 (0 macros)
Sequential cells (weight register file)	12,211 (~10,816 are the weight memory)
Routed wirelength / vias	3.73 m / 523,861

Detailed routing converged cleanly — routing DRC fell from 34,567 violations to zero over seven iterations — and physical verification passed end to end: **0 DRC errors** (Magic and KLayout), **0 LVS errors**, 0 power-grid violations, and a worst-case IR drop of ~15.7 mV. The flow produced a final GDS. Timing was evaluated across process corners:

Corner	Setup WNS	Hold WNS	Status
Typical (tt, 25°C, 1.80V)	+9.13 ns	+0.31 ns	clean
Fast (ff, -40°C, 1.95V)	+13.62 ns	+0.11 ns	clean

Slow (ss, 100°C, 1.60V)	-5.26 ns (133 ep)	-0.34 ns (139 ep)	violations
-------------------------	-------------------	-------------------	------------

The design closes with large positive slack at the typical and fast corners at 40 MHz but fails setup at the slow corner by 5.26 ns; the first-order all-corner closing frequency is therefore about 33 MHz. The signoff also reports unrepaired design-rule violations (826 max-fanout, ~1,800 max-slew). Both the slow-corner failure and the DRVs trace to the register-file weight memory and the fact that the post-global-route resizer was disabled in the area-optimized configuration (Section 9). Measured total power is 0.269 W (0.119 W internal, 0.150 W switching, ~1.2 μ W leakage).

8. Benchmark Results

The accelerator was benchmarked against the M1 software baseline (Python + NumPy on an x86-64 CPU). Hardware figures are measured: the 217-cycle latency is from the co-simulation, and the frequency, power, and energy come from the completed signoff at the 40 MHz typical-corner operating point. One inference takes $217 \times 25 \text{ ns} = 5.43 \mu\text{s}$, for a throughput of ~184,000 inferences/s and a compute rate of ~490 MFLOPS; at 0.269 W each inference costs ~1.46 μ J, an efficiency of ~1.8 GFLOPS/W.

Metric	SW baseline (CPU)	HW accelerator
Single-sample latency	12.0 μs	5.43 μs (2.2 $\times$ faster)
Batch throughput (10k)	6,355,000 /s	184,332 /s
Compute rate	16,885 MFLOPS	490 MFLOPS
Memory footprint	1.2 MB	1,385 B (928 $\times$ less)
F1 score	0.661 (FP32)	0.672 (INT8+QAT)
Total power	~5-15 W (est.)	0.269 W (measured)
Energy / inference	~60-180 μ J (est.)	1.46 μ J (measured)

The accelerator wins decisively on the metrics that matter for the edge use case: single-sample latency (2.2 $\times$), memory footprint (928 $\times$), and energy per inference (~80 $\times$ lower, with the hardware side now measured rather than estimated). The software baseline wins only on batch throughput, because NumPy uses SIMD to process thousands of samples at once as a batched GEMM — a different kernel than the single-sample matrix-vector product the hardware performs, and one irrelevant to a real-time stream of individual sensor readings. Completing the physical flow also corrected the energy story: earlier milestones could only estimate ~5-15 mW and a ~4,600 $\times$ energy advantage, whereas the measured 0.269 W — driven by the register-file memory and its read-mux fabric switching every cycle — gives a realistic ~80 $\times$. All figures are at the nominal 40 MHz operating point; a conservative all-corner rating at ~33 MHz scales latency to ~6.6 μs .

9. What Did Not Work

A signed/unsigned requantization bug. In the bias/requantization lane the accumulator was right-shifted ($\text{ACC} \ggg 7$) and combined with a bias term, but an unsigned concatenation operand made the whole expression unsigned, silently turning the intended arithmetic shift into a logical one. Negative pre-activations then saturated to 127 instead of going through ReLU to zero — one neuron’s output inverted while a neighbour stayed correct, the tell-tale signature of a shared path with conditional sign handling. It was found by comparing against the Python reference and fixed by wrapping the operands in signed casts. A related sign-extension fix was applied to the MSE difference path.

A layer-transition race. Activations for the next layer could be overwritten before they were fully consumed; this was resolved by inserting the S_COPY state, which adds four cycles per inference (213 → 217) in exchange for correct hand-off between layers.

The 83 MHz target was infeasible. At a 12 ns constraint, post-CTS worst slack was -8.6 ns and global routing failed with excessive congestion, because chasing the impossible target made the resizer insert ~30,000 timing-repair buffers that overwhelmed the routing channels. Relaxing to 25 ns and enlarging the die resolved both.

The back-end flow exceeded free-tier memory. Repeated runs on a ~13 GB environment ran cleanly through global routing but were killed at the start of post-route STA, the most memory-intensive stage. The flow completed only after migrating to a 29 GB environment; the practical lesson is that signoff for a register-file-heavy design of this size needs roughly 29 GB.

The completed signoff exposed a slow-corner timing gap and DRVs. The design fails setup at the slow corner (-5.26 ns) and carries fanout/slew violations. Both stem from the same root cause: the weight memory implemented as a flat register file. Reading one of 1,352 entries per cycle synthesizes into a ~26,000-cell multiplexer fabric that creates long, high-fanout read paths, left unrepaired because the post-global-route resizer was disabled in the area-optimized configuration. The principled remedy — and the primary direction for future work — is to replace the register file with a **compiled SRAM macro** (e.g. via OpenRAM): a single dense hard block in place of ~37,000 cells would cut area, eliminate the high-fanout read paths driving the DRVs, and shorten the critical path enough to recover slow-corner margin. Re-enabling post-route timing optimization would clean the residual violations on the existing design.

All synthesis, timing, and power figures are taken from the completed OpenLane 2 signoff metrics; benchmark cycle counts are from Icarus Verilog co-simulation. Figures 1–2 are schematic; Figures 3–4 are generated from the measured roofline data and the simulation VCD.